

Machine Learning Project

Kevin Sony

Summer 2025

1 Introduction

We want to create a simple machine learning library in C. This is to both relearn how C works, and to fully understand the mathematics behind machine learning.

There are some things we want from this library. It should be a dynamic library so that wrappers can be written in Python, Java, and other languages. There should be an intuitive way to construct these wrappers, so that it does not get too complicated to implement. Finally, the library itself should be intuitive to read and build, so that others can potentially debug if necessary.

Some things to hopefully implement:

- Linear/Logistic Regression
- K-Nearest Neighbors
- K-Means Clustering
- Artificial Neural Networks

The basic premise of machine learning algorithms is for some given inputs, we want to find out some information from this output, whether it be a classification or a prediction. Specifically, machine learning algorithms construct a function f that should take in an input x from a specific input space, and return the y we expect to see.

For example, if we wanted to classify images as animals, humans, cars, and other objects, the function's domain is the space of images of all the objects that are in the classifier, and the function's range is a probability vector, containing the probabilities it is any one of the specified objects. So if we pass in an image of a dog, we want our neural network to return a vector indicating the object in the image has a high probability of being a dog, and a low probability of being anything else in the classifier.

To be able to construct such a function, we first need to find some data to construct it from. From there, we check its validity by testing on other data not used to construct it.

2 Artificial Neural Networks

ANNs construct the function f through applying affine then nonlinear transformations multiple times.

We start with an input $x^{(0)} \in \mathbb{R}^{m_0}$ (vectors are represented vertically), apply an affine transformation to get $z^{(1)} \in \mathbb{R}^{m_1}$ and nonlinear transform to turn it to an output $y^{(1)} \in \mathbb{R}^{m_1}$. This then becomes the new input $x^{(1)} = y^{(1)}$. By applying this transformation n times, we get the output $y^{(n)} \in \mathbb{R}^{m_n}$. In other words,

$$\begin{aligned}y^{(1)} &= a^{(1)}(W^{(1)}x^{(0)} + b^{(1)}) \\y^{(2)} &= a^{(2)}(W^{(2)}x^{(1)} + b^{(2)}) \\&\vdots \\y^{(n)} &= a^{(n)}(W^{(n)}x^{(n-1)} + b^{(n)})\end{aligned}$$

where $W^{(1)}, \dots, W^{(n)}$ are weights, $b^{(1)}, \dots, b^{(n)}$ are biases, and $a^{(1)}, \dots, a^{(n)}$ are nonlinear "activation" functions. This implicitly defines a function $\hat{f}$, where $\hat{f}(x_1) = y_n$.

It doesn't do much good to simply have a function $\hat{f}$. We want $y^{(n)}$ to equal a true value t , i.e. we want to find a function f such that $f(x^{(0)}) = t$. If $\hat{f} = f$, we are done, but if not, we need to modify $\hat{f}$ to get closer to f , and this is done by modifying its parameters, the weights and biases.

There are several ways to solve for the optimal weights, and all of them involve trying to minimize a loss function L that takes in $y^{(n)}$ and t and computes a measure of how far off the two vectors are. From there, one can either try minimizing the loss function through various methods such as gradient descent or Newton's method. They each have various trade-offs. Newton's method converges quadratically, but it requires convexity (at the very least, local convexity if not global), and the Hessian can be quite computationally expensive to compute. Gradient descent converges linearly, but it is much cheaper to compute gradients.

2.1 Backpropagation via Gradient Descent

Different loss functions are optimal for different tasks (cross-entropy for classification, mean square error for regression). Gradient descent is frequently employed to solve for the optimal weights and biases. The various gradients of the loss function with respect to many the weights and biases are used to update said weights and biases.

To derive the gradients necessary to update the weights and biases, we must first start off with an appropriate loss function, L . The derivative of L with

respect to $y^{(n)}$ is

$$\frac{\partial L}{\partial y^{(n)}} = \nabla_{y^{(n)}} L = \begin{bmatrix} \frac{\partial L}{\partial y_1^{(n)}} \\ \frac{\partial L}{\partial y_2^{(n)}} \\ \vdots \\ \frac{\partial L}{\partial y_{m_n}^{(n)}} \end{bmatrix}$$

Let $z^{(n)} = W^{(n)}x^{(n-1)} + b^{(n)}$. Then $y^{(n)} = a^{(n)}(z^{(n)})$. Note that because the activation function a_n is applied to each entry individually, each entry in the inputs for a_n is independent of the other. In other words, it is a diagonal matrix.

$$\frac{\partial y^{(n)}}{\partial z^{(n)}} = \text{diag} \left(\frac{\partial y_1^{(n)}}{\partial z_1^{(n)}}, \dots, \frac{\partial y_{m_n}^{(n)}}{\partial z_{m_n}^{(n)}} \right)$$

We also know that

$$\begin{aligned} \frac{\partial z^{(n)}}{\partial W^{(n)}} &= x^{(n-1)} \\ \frac{\partial z^{(n)}}{\partial b^{(n)}} &= I \end{aligned}$$

Combining the results, we get

$$\frac{\partial L}{\partial W^{(n)}} = \frac{\partial L}{\partial y^{(n)}} \frac{\partial y^{(n)}}{\partial z^{(n)}} \frac{\partial z^{(n)}}{\partial W^{(n)}} = \frac{\partial y^{(n)}}{\partial z^{(n)}} \nabla_{y^{(n)}} L \left(x^{(n-1)} \right)^T$$

$$\frac{\partial L}{\partial b^{(n)}} = \frac{\partial L}{\partial y^{(n)}} \frac{\partial y^{(n)}}{\partial z^{(n)}} \frac{\partial z^{(n)}}{\partial b^{(n)}} = \frac{\partial y^{(n)}}{\partial z^{(n)}} \nabla_{y^{(n)}} L$$

Finally, we look at how $x^{(n-1)}$ "changes", since it is also the output to the previous layers, and seeing how to change it will result in changes propagating outwards

$$\frac{\partial L}{\partial x^{(n-1)}} = \frac{\partial L}{\partial y^{(n)}} \frac{\partial y^{(n)}}{\partial z^{(n)}} \frac{\partial z^{(n)}}{\partial x^{(n-1)}} = \left(W^{(n)} \right)^T \left(\frac{\partial y^{(n)}}{\partial z^{(n)}} \nabla_{y^{(n)}} L \right)$$

We can generalize this result to the inner layer weights and layers

$$\frac{\partial L}{\partial W^{(i)}} = \frac{\partial L}{\partial y^{(i)}} \frac{\partial y^{(i)}}{\partial z^{(i)}} \frac{\partial z^{(i)}}{\partial W^{(i)}} = \frac{\partial y^{(i)}}{\partial z^{(i)}} \frac{\partial L}{\partial x^{(i)}} \left(x^{(i-1)} \right)^T$$

$$\frac{\partial L}{\partial b^{(i)}} = \frac{\partial L}{\partial y^{(i)}} \frac{\partial y^{(i)}}{\partial z^{(i)}} \frac{\partial z^{(i)}}{\partial b^{(i)}} = \frac{\partial y^{(i)}}{\partial z^{(i)}} \frac{\partial L}{\partial x^{(i)}}$$

$$\frac{\partial L}{\partial x^{(i-1)}} = \frac{\partial L}{\partial y^{(i)}} \frac{\partial y^{(i)}}{\partial z^{(i)}} \frac{\partial z^{(i)}}{\partial x^{(i-1)}} = \left(W^{(i)} \right)^T \left(\frac{\partial y^{(i)}}{\partial z^{(i)}} \frac{\partial L}{\partial x^{(i)}} \right)$$

Note that $x^{(i)} = y^{(i)}$, and so

$$\frac{\partial L}{\partial x^{(i)}} = \frac{\partial L}{\partial y^{(i)}} = \nabla_{y^{(i)}} L$$

2.2 Various Optimizing

While these formulas give the gradients, there is of course optimizations which can be employed, and the first step to optimizing is to notice where optimizations can be applied.

One thing to consider is that $\frac{\partial y^{(n)}}{\partial z^{(n)}} \nabla_{y^{(n)}} L$ and $\frac{\partial y^{(i)}}{\partial z^{(i)}} \frac{\partial L}{\partial x^{(i)}}$ show up together many times, so it would be more efficient to compute it once rather than many times. Defining $\delta^{(n)}$ and $\delta^{(i)}$

$$\delta^{(n)} = \frac{\partial y^{(n)}}{\partial z^{(n)}} \nabla_{y^{(n)}} L$$

$$\delta^{(i)} = \frac{\partial y^{(i)}}{\partial z^{(i)}} \frac{\partial L}{\partial x^{(i)}} = \frac{\partial y^{(i)}}{\partial z^{(i)}} \frac{\partial L}{\partial y^{(i)}} = \frac{\partial y^{(i)}}{\partial z^{(i)}} \nabla_{y^{(i)}} L$$

we get the following equations for the output layer

$$\frac{\partial L}{\partial W^{(n)}} = \delta^{(n)} \left(x^{(n-1)} \right)^T$$

$$\frac{\partial L}{\partial b^{(n)}} = \delta^{(n)}$$

$$\frac{\partial L}{\partial x^{(n-1)}} = \left(W^{(n)} \right)^T \delta^{(n)}$$

and for the hidden layers

$$\frac{\partial L}{\partial W^{(i)}} = \delta^{(i)} \left(x^{(i-1)} \right)^T$$

$$\frac{\partial L}{\partial b^{(i)}} = \delta^{(i)}$$

$$\frac{\partial L}{\partial x^{(i-1)}} = \left(W^{(i)} \right)^T \delta^{(i)}$$

Since $\frac{\partial L}{\partial y^{(i-1)}} = \frac{\partial L}{\partial x^{(i-1)}}$, the formula for $\delta^{(i-1)}$ is

$$\delta^{(i-1)} = \frac{\partial L}{\partial y^{(i-1)}} \frac{\partial y^{(i-1)}}{\partial z^{(i-1)}} = \left(\left(W^{(i)} \right)^T \delta^{(i)} \right)^T \frac{\partial y^{(i-1)}}{\partial z^{(i-1)}}$$

This reformulation is much more compact, and it opens the door for more optimization. In particular, $\frac{\partial y^{(i)}}{\partial z^{(i)}}$ is a diagonal matrix, so zeros populate all nondiagonal entries. It is unnecessary to store all those extra zeros, and it is enough to just store it as a vector, $a'_i(z^{(i)})$

$$a'_i(z^{(i)}) = \begin{bmatrix} \frac{\partial y_1^{(i)}}{\partial z_1^{(i)}} \\ \vdots \\ \frac{\partial y_{m_i}^{(i)}}{\partial z_{m_i}^{(i)}} \end{bmatrix}$$

This change results in slight modifications to the existing formulas. For one, with this change, rather than going through the full matrix multiplication to evaluate $\delta^{(i)}$, one can simply multiply the corresponding entries together.

$$\delta^{(i)} = a'_i(z^{(i)}) \odot \nabla_{y^{(i)}} L$$

The formula for $\frac{\partial L}{\partial W^{(i)}}$ and $\frac{\partial L}{\partial b^{(i)}}$ remains the same. The formula for $\delta^{(i-1)}$ becomes

$$\delta^{(i-1)} = \left(\left(W^{(i)} \right)^T \delta^{(i)} \right) \odot a'_{i-1}(z^{(i-1)})$$

The rules for updating are

$$\begin{aligned} W^{(i)} &\leftarrow W^{(i)} - \gamma \frac{\partial L}{\partial W^{(i)}} \\ b^{(i)} &\leftarrow b^{(i)} - \gamma \frac{\partial L}{\partial b^{(i)}} \end{aligned}$$

2.3 Code

There are two ways to approach the construction of the artificial neural network. One is to use a set of implicit layers by only containing a set of matrices and vectors. This approach is relatively simple to set up, requiring only minimal set up.

```

1 struct ann_ {
2     matrix** weights;
3     vector** biases;
4     activation_func* acc;
5
6     size_t* layers;
7     size_t number_of_layers;
8     number gamma;
9
10    boolean is_classifier;
11 };
12 typedef struct ann_ ann;
13

```

Listing 1: Artificial Neural Network in C

A Chain Rule for functions mapping to and from multiple dimensions

The chain rule, while quite powerful, can also be insanely unintuitive. For example, if one was to take the chain rule for

$$\frac{\partial L}{\partial W^{(n)}} = \frac{\partial L}{\partial y^{(n)}} \frac{\partial y^{(n)}}{\partial z^{(n)}} \frac{\partial z^{(n)}}{\partial W^{(n)}}$$

they immediately run into a problem. $\frac{\partial L}{\partial y^{(n)}}$ has dimension $m_n \times 1$, $\frac{\partial y^{(n)}}{\partial z^{(n)}}$ has dimension $m_n \times m_n$, and finally, $\frac{\partial z^{(n)}}{\partial W^{(n)}}$ has dimension $m_n \times m_{n-1}$. Simple matrix multiplication does not work.

The trick is to work backwards. First note that $\frac{\partial L}{\partial W^{(n)}}$ is a matrix, meaning it can be indexed by two values, i and j . Using the chain rule, we get

$$\frac{\partial L}{\partial W_{i,j}^{(n)}} = \sum_{k,l} \left(\frac{\partial L}{\partial y_k^{(n)}} \frac{\partial y_k^{(n)}}{\partial z_l^{(n)}} \frac{\partial z_l^{(n)}}{\partial W_{i,j}^{(n)}} \right)$$

Let $A = \frac{\partial L}{\partial y^{(n)}}$, $B = \frac{\partial y^{(n)}}{\partial z^{(n)}}$, and $C = \frac{\partial z^{(n)}}{\partial W^{(n)}}$. Since A and B are matrices,

$A_{1,k} = \frac{\partial L}{\partial y_k^{(n)}}$ and $B_{k,l} = \frac{\partial y_k^{(n)}}{\partial z_l^{(n)}}$. The case of C is problematic since C is indexed

by three numbers, i.e. $C_{l,i,j} = \frac{\partial z_l^{(n)}}{\partial W_{i,j}^{(n)}}$. To resolve this, we must eliminate one of the indices. Recall that

$$z^{(n)} = W^{(n)}x^{(n-1)} + b^{(n)}$$

That means

$$z_l^{(n)} = \sum_j W_{l,j}^{(n)} x_j^{(n-1)} + b_l^{(n)}$$

and so

$$\frac{\partial z_l^{(n)}}{\partial W_{i,j}^{(n)}} = \begin{cases} x_j^{(n-1)} & \text{if } i = l \\ 0 & \text{if } i \neq l \end{cases}$$

$$\frac{\partial z_l^{(n)}}{\partial W_{i,j}^{(n)}} = \delta_{l,i} \cdot x_j^{(n-1)}$$

where $\delta_{l,i}$ is the Kronecker delta. Putting together the results, we get

$$\frac{\partial L}{\partial W_{i,j}^{(n)}} = \sum_{k,l} \left(A_{1,k} B_{k,l} \delta_{l,i} \cdot x_j^{(n-1)} \right)$$

and since $\delta_{l,i} = 0$ except when $l = i$, we get

$$\frac{\partial L}{\partial W_{i,j}^{(n)}} = \sum_{k,l} \left(A_{1,k} B_{k,i} \cdot x_j^{(n-1)} \right)$$

The formula which satisfies this formula for all indices is

$$\frac{\partial L}{\partial W^{(n)}} = \frac{\partial y^{(n)}}{\partial z^{(n)}} \nabla_{y^{(n)}} L \left(x^{(n-1)} \right)^T$$

B Derivation of optimization

To make the neural network more efficient, the formula

$$\delta^{(i)} = \frac{\partial y^{(i)}}{\partial z^{(i)}} \nabla_{y^{(i)}} L$$

was introduced. This is a matrix multiplication, and thus

$$\delta_j^{(i)} = \sum_k \frac{\partial y_k^{(i)}}{\partial z_j^{(i)}} (\nabla_{y^{(i)}} L)_j$$

The activation function only acts on each entry and so

$$\frac{\partial y_k^{(i)}}{\partial z_j^{(i)}} = 0 \text{ if } k \neq j$$

Therefore,

$$\delta_j^{(i)} = \frac{\partial y_j^{(i)}}{\partial z_j^{(i)}} (\nabla_{y^{(i)}} L)_j$$